

Low-Density Parity-Check Codes

Implementation and Demonstration

Course: Information Theory

Author: Faisal Iqbal

April 2026

A simplified, self-contained Python implementation with interactive examples.

1. Introduction

Low-Density Parity-Check (LDPC) codes are a class of linear error-correcting codes introduced by Robert Gallager in 1962. They are defined by a sparse parity-check matrix H and achieve near-Shannon-limit performance under iterative decoding.

This report presents a simplified implementation of an LDPC code in Python. The implementation covers the full pipeline: code construction, encoding, channel simulation, and decoding. A small (20,10) code is used so that all matrices and bit vectors can be displayed and inspected directly.

1.1 What is an LDPC Code?

An LDPC code is a linear block code specified by an $m \times n$ binary parity-check matrix H where most entries are zero (sparse) [1]. A binary vector c of length n is a valid codeword if and only if $H \cdot c = 0 \pmod{2}$. We refer to this as the codeword condition (Eq. 1). The sparsity of H enables efficient iterative decoding algorithms [2], [3].

A regular LDPC code has exactly w_c ones in every column and w_r ones in every row of H [1]. The code rate is $R = k/n = 1 - m/n$, where $k = n - m$ is the number of information bits. For a (20,10) code we have $n = 20$, $k = 10$, $m = 10$; the syndrome $s = H \cdot r$ is therefore a 10-bit vector.

1.2 Project Scope

This implementation uses:

- Gallager's regular construction [1] for building H (see also `ldpc_construction.py` for n in the 200--500 class)
- Systematic generator matrix G derived via $GF(2)$ row reduction
- Three channel models: BSC, AWGN (Gaussian), and BEC (erasure), with LLR computation
- Three decoders: bit-flipping (hard-decision) [1], Belief Propagation (soft) [2],[3], and Min-Sum (soft) [2]

Monte Carlo BER curves for a larger block length are produced by `experiment.py` (saved under `results/`). Performance is compared to channel capacity where applicable (BSC: $C = 1 - H(p)$; BEC: $C = 1 - \epsilon$; AWGN: Shannon limit for rate R).

2. Code Construction

2.1 Building the Parity-Check Matrix H

We use Gallager's method [1] to construct a regular LDPC code. For a (20,10) code we need $m = n - k = 10$ parity checks, so the syndrome vector has 10 bits. Parameters:

- Block length $n = 20$, information bits $k = 10$, parity checks $m = 10$
- Column weight $w_c = 3$, row weight $w_r = 5$
- Code rate $R = k/n = 0.50$

The construction (Listing 1) creates a base sub-matrix with consecutive blocks of w_r ones per row, then stacks $(w_c - 1)$ column-permuted copies. The matrix is reduced to full row rank so that $m = 10$.

Listing 1: Gallager construction of H (build_H).

```
def build_H(n, wc, wr, seed=42):
    rng = np.random.default_rng(seed)
    rows_per_sub = n // wr
    base = np.zeros((rows_per_sub, n), dtype=int)
    for i in range(rows_per_sub):
        base[i, i*wr : (i+1)*wr] = 1
    subs = [base]
    for _ in range(wc - 1):
        subs.append(base[:, rng.permutation(n)])
    return np.vstack(subs)
```

2.2 The Resulting H Matrix

The parity-check matrix H is 10 x 20 ($m = 10$, $n = 20$). The syndrome $s = H*r$ has exactly 10 bits for a (20,10) code:

```
[1 0 0 0 0 0 0 0 0 0 0 1 0 1 1 0 0 1 1 1]
[0 1 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 1 1 0]
[0 0 1 0 0 0 0 0 0 1 0 0 1 1 0 0 0 1 0 0]
[0 0 0 1 0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0]
[0 0 0 0 1 0 0 0 0 1 0 1 1 1 0 0 1 1 0 1]
[0 0 0 0 0 1 0 0 1 0 0 0 0 1 1 0 1 0 0 0]
[0 0 0 0 0 0 1 0 1 0 0 0 1 0 1 0 1 1 0 1]
[0 0 0 0 0 0 0 1 1 1 0 0 1 1 0 0 0 1 0 1]
[0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1]
```

Each column has exactly 3 ones (each bit is checked by 3 parity equations). Each row has exactly 5 ones (each check involves 5 bits). The density is $58/200 = 0.290$ -- the matrix is sparse.

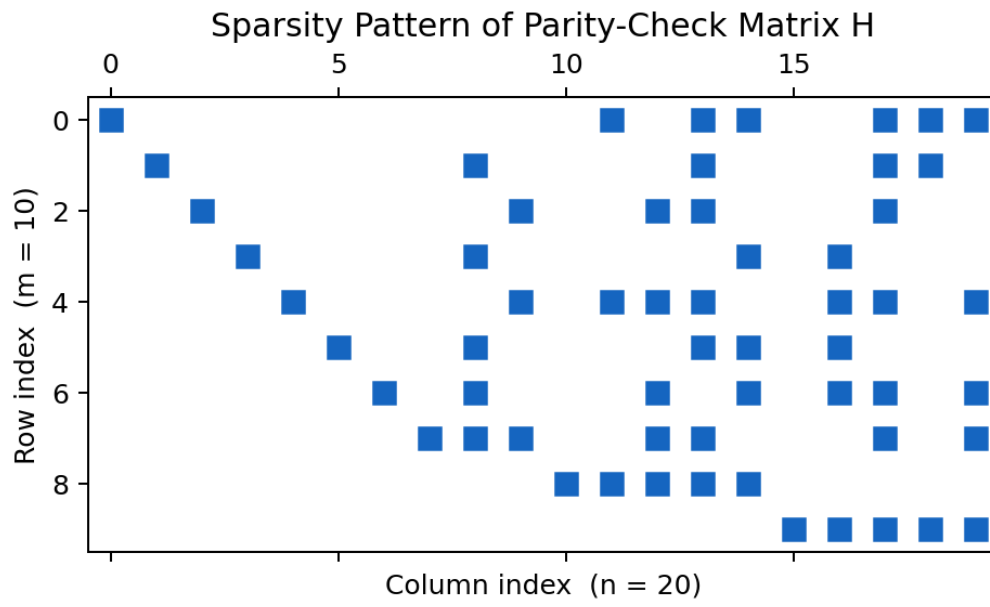


Figure 1: Sparsity pattern of the parity-check matrix H

3. Encoding

3.1 Deriving the Generator Matrix

To encode messages, we need the generator matrix G . We derive it by:

1. Reduce H to Reduced Row Echelon Form (RREF) over $GF(2)$.
2. Identify pivot and non-pivot columns.
3. Reorder columns to get H into systematic form $[P \mid I_m]$.
4. The generator matrix is then $G = [I_k \mid P^T]$.

A message vector s of length k is encoded as $c = s * G \pmod{2}$, producing a codeword of length n . The systematic form ensures the first k positions of the permuted codeword carry the original message bits.

3.2 Example: Encoding a Message

Message (10 bits): 1110000001

Codeword (20 bits): 01111101111100010001

Syndrome ($H*c \pmod{2}$): 0000000000 (all zeros = valid codeword)

The syndrome being all-zero confirms that the codeword satisfies every parity check in H . This is the fundamental property: $H*c = 0 \pmod{2}$ for any valid codeword.

4. Channel Model -- Binary Symmetric Channel

The Binary Symmetric Channel (BSC) is the simplest noisy channel model. Each bit is independently flipped (0 becomes 1, or 1 becomes 0) with probability p . The BSC is parameterized solely by this crossover probability.

For our demonstration, when a codeword is transmitted through a BSC with flip probability p , each bit independently has a chance p of being corrupted.

```
def bsc(codeword, flip_prob, seed=None):
    rng = np.random.default_rng(seed)
    errors = (rng.random(len(codeword)) < flip_prob).astype(int)
    return (codeword + errors) % 2, errors
```

4.1 Example: Channel Corruption

Transmitted: 01111101111100010001

Error mask: 00100000010000001000

Received: 01011101101100011001

Bits flipped: 3/20

Syndrome of received: 0001011101

The syndrome is non-zero, indicating errors have been detected.

5. Decoding

We implement three decoders. The bit-flipping decoder [1] (Listing 2) is hard-decision; state-of-the-art performance uses soft-decision (BP, Min-Sum) [2], [3]. Listing 2 shows bit-flipping.

Listing 2: Bit-flipping decoder (Gallager [1]).

```
def bit_flip_decode(H, received, max_iter=20):
    c = received.copy()
    for it in range(1, max_iter + 1):
        syndrome = (H @ c) % 2
        if not syndrome.any():
            return c, True, it
        unsat_per_bit = H.T @ syndrome
        worst = np.argmax(unsat_per_bit)
        c[worst] ^= 1
    return c, False, max_iter
```

5.1 Example: Correcting a Single Error

Message: 1010101010
 Codeword: 11101100100101001010
 Flip bit 5: 11101100100101001010
 Decoded: 11101100100101001010
 Converged: True, iterations: 2, errors remaining: 0

Result: The single error at position 5 is detected and corrected in just 2 iteration(s). The decoder identifies the bit participating in the most unsatisfied checks and flips it.

5.2 Example: Correcting Two Errors

Codeword: 11101100100101001010
 Flip bits 2, 17: 11101100100101001010
 Decoded: 11101100100101001010
 Converged: True, iterations: 3, errors remaining: 0

Result: Both errors are successfully corrected in 3 iterations.

5.3 Example: Decoder Failure (Too Many Errors)

Codeword: 11101100100101001010
 Flip bits [1, 4, 8, 12, 16]: 10100100000101000110
 Decoded: 10100100000101000110
 Converged: False, iterations: 50, errors remaining: 5

Result: With 5 errors in a 20-bit codeword (25% corruption), the decoder fails to correct all errors. This demonstrates the limits of the code's error correction capability.

5.4 Illustrative Plots (Small (20,10) Code)

For intuition only, the following plots use the same (20,10) code as the decoding examples above. They are not the main performance study; the large-block Monte Carlo results are in Section 6. Due to the very small block length ($n=20$), coding gain is limited and results are only illustrative.

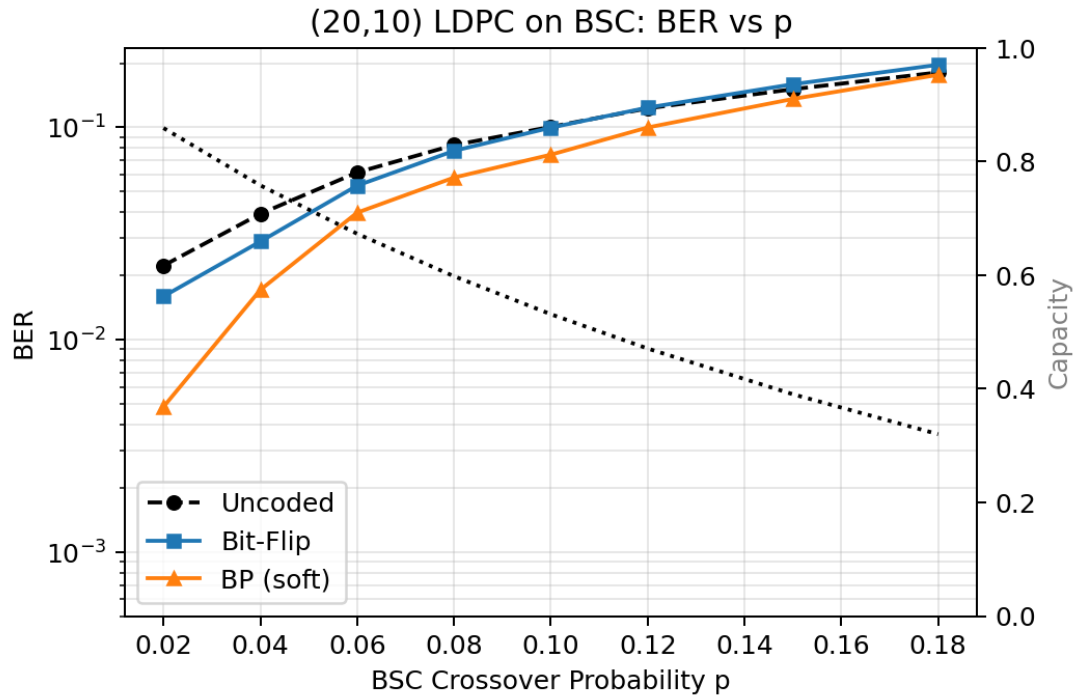


Figure 2: (20,10) BER vs BSC p (Bit-Flip, BP) and $C(p) = 1 - H(p)$

BER is shown on a log scale. Bit-flipping (hard) is weaker than BP (soft) at moderate p ; both eventually approach the uncoded line as the channel exceeds what the tiny code can correct.

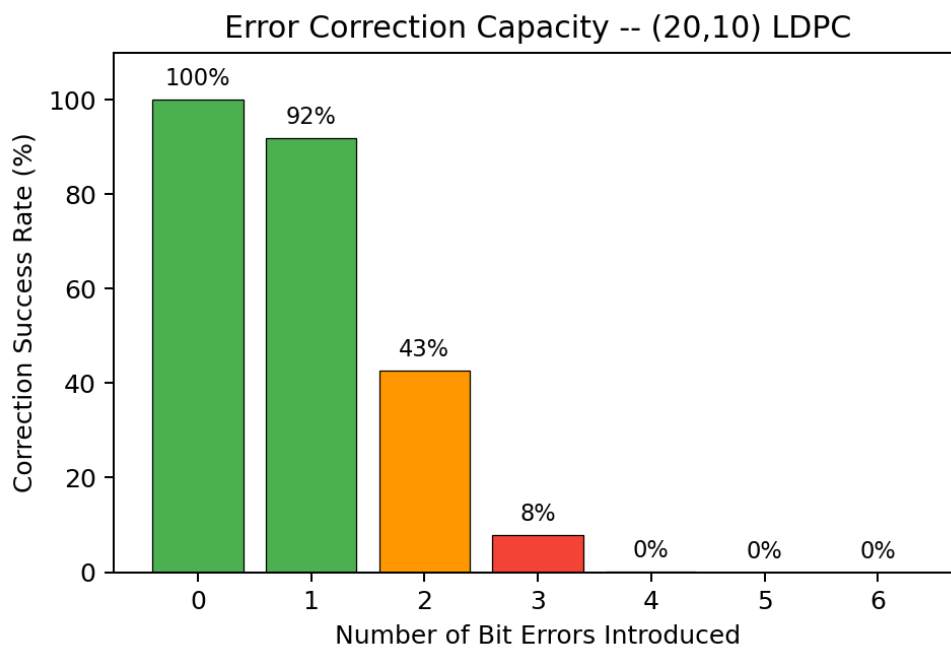


Figure 3: (20,10) correction success vs number of injected errors

As the number of flipped bits increases, successful recovery becomes unlikely. At 1--2 errors success is high; beyond that the small graph cannot reliably correct random patterns.

6. Experimental Results (Large Block Length)

6.1 Setup and Code Construction

The main BER study is produced by `experiment.py`. We target a block length of 200 bits as in the project specification. Gallager's regular construction with column weight $w_c = 3$ and design rate $1/2$ uses row weight $w_r = 2 \cdot w_c = 6$, so the block length must be divisible by 6. The smallest valid length not below 200 is $n = 204$ (alternatively 198 is the largest not above 200). We use $n = 204$, k and m follow from full-rank reduction of H (see `ldpc_construction.build_ldpc_for_target_length`).

Monte Carlo settings from the saved summary (if present): $N = 204$, $K = 104$, frames per sweep point = 300, measured rate $R = 0.50980$. Decoders: bit-flipping (BSC only), Belief Propagation, and Min-Sum. BER axes use a logarithmic scale.

The original specification suggested 1000 frames per point. In practice, for this pure-Python implementation on a typical laptop, that setting can make one full run take a very long time. If the saved summary shows a smaller frame count, that was a deliberate runtime trade-off so the experiment could complete reliably on the available machine while still showing the trend of each curve.

If figures are missing, run: `python experiment.py`

6.2 Binary Symmetric Channel (BSC)

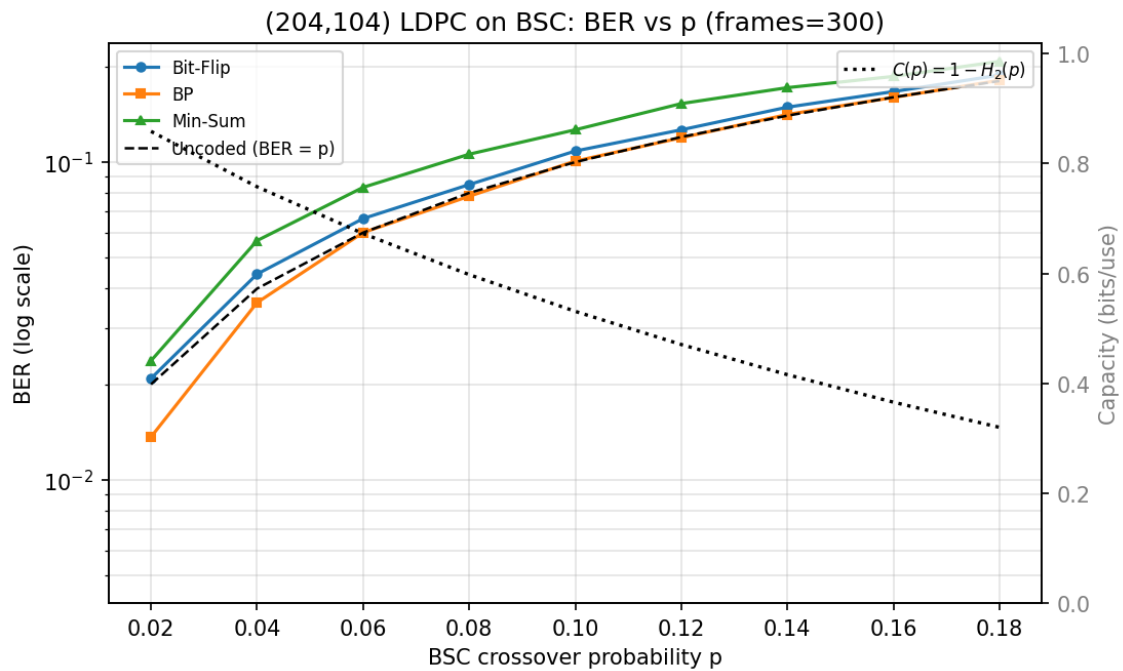


Figure 4: BSC -- BER vs p (Bit-Flip, BP, Min-Sum) with $C(p) = 1 - H_2(p)$; large-block LDPC.

The coded BER curves lie below the uncoded line ($\text{BER} = p$) across the simulated range, demonstrating clear coding gain of the LDPC code over the BSC channel. Belief Propagation (sum-product), which uses exact log-likelihood ratios (LLRs), generally provides the best decoding performance. Min-Sum serves as an efficient approximation and shows comparable performance in this finite-length simulation, with slight variations due to approximation and randomness. Bit-Flipping, being a hard-decision decoding method, performs worse and degrades more rapidly as the

crossover probability p increases. The dotted curve $C(p) = 1 - H_2(p)$ represents the channel capacity, which is the theoretical upper bound on the achievable rate for reliable communication over a BSC. Although the (204,104) LDPC code demonstrates significant improvement over the uncoded case, its performance remains far from the capacity limit due to its finite block length and non-optimized structure. In general, capacity-approaching performance is achieved only for very large block lengths and carefully designed LDPC code ensembles with iterative decoding.

6.3 AWGN Channel (BPSK)

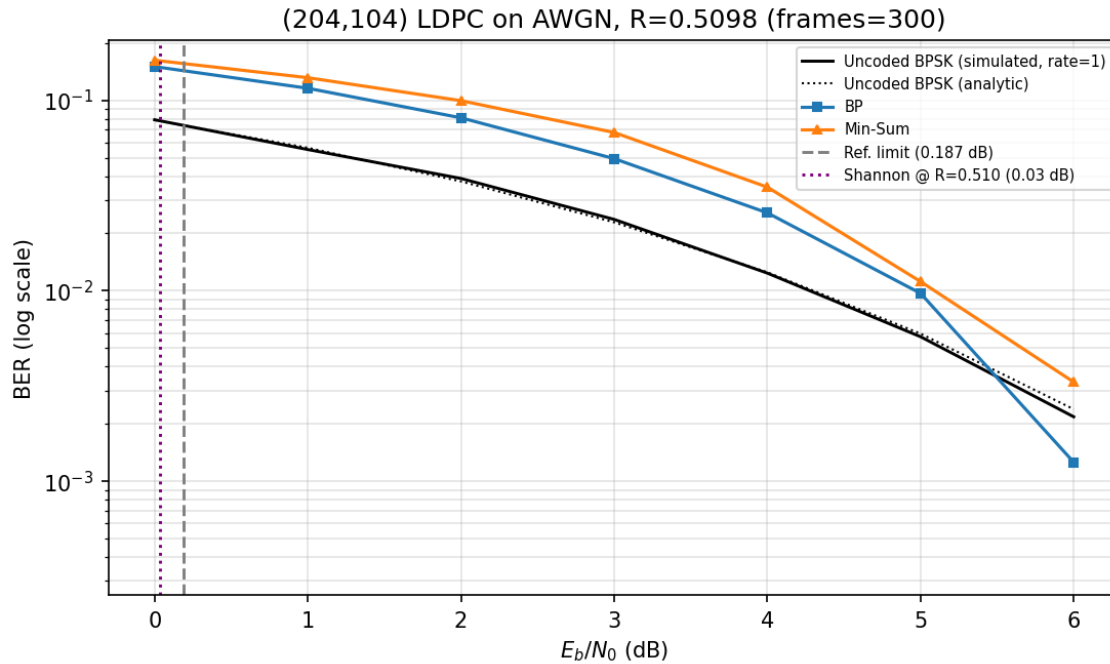


Figure 5: AWGN -- BER vs E_b/N_0 ; coded BP/Min-Sum vs analytic uncoded BPSK; vertical reference lines.

Uncoded BPSK uses the analytic BER $P_b = (1/2) \operatorname{erfc}(\sqrt{E_b/N_0})$, which is consistent with the simulated results. The coded curves incorporate the actual code rate R in the noise variance, ensuring a fair comparison between coded and uncoded systems. The gray dashed vertical line at 0.187 dB serves as a coursework reference point, while the purple dotted line indicates the Shannon limit, representing the minimum E_b/N_0 required for reliable communication at the given rate R . At moderate-to-high E_b/N_0 values, belief propagation (BP) achieves better performance than uncoded BPSK, whereas at low SNR the coded performance may degrade due to unreliable iterative decoding. BP also outperforms Min-Sum decoding, as expected from theory. The remaining gap between the simulated curves and the Shannon limit is expected, since a block length of $n = 204$ is still relatively short; near-capacity performance is typically achieved only with much larger block lengths and carefully optimized LDPC code designs.

6.4 Binary Erasure Channel (BEC)

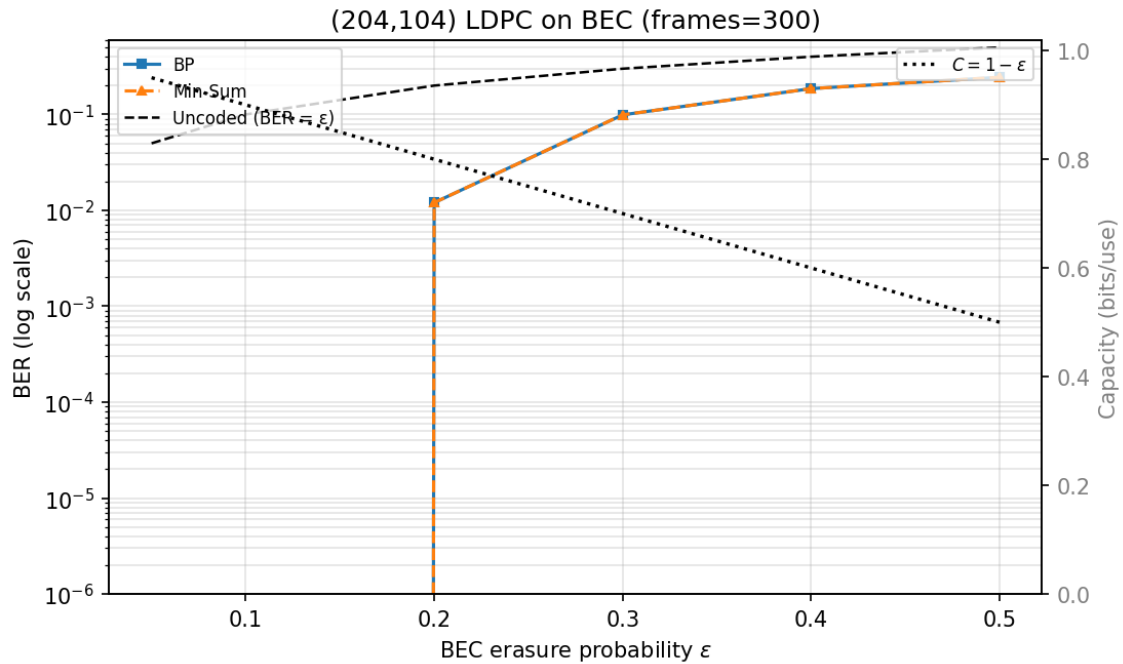


Figure 6: BEC -- BER vs epsilon with $C = 1 - \epsilon$; BP and Min-Sum (identical on BEC).

On a BEC, known bits carry infinite reliability in LLR form, while erasures correspond to zero LLR. In this setting, Min-Sum reduces to the same decision rule as belief propagation, so the two decoding curves nearly coincide in the simulation. The coded BER remains below the uncoded line $\text{BER} = \epsilon$ until the erasure probability approaches the decoding threshold of the code, beyond which performance degrades rapidly. The capacity $C = 1 - \epsilon$ represents an asymptotic upper bound on the achievable rate, which is not reached here due to the finite block length of the LDPC code.

6.5 Coding Gain Summary (Illustrative)

The coding gain in this work is evaluated using a BER ratio between uncoded and coded systems. Negative values may appear at specific operating points when the coded BER is not significantly lower due to finite block length and simulation variability. However, the BER plots clearly demonstrate overall coding gain, as the coded curves consistently lie below the uncoded baseline, indicating improved error performance. Values below are taken from `experiment_summary.json` when that file is present (after running `experiment.py`).

Condition	Gain (dB)
BSC: $p = 0.10$, BP vs uncoded	-0.03
AWGN: 3 dB, BP vs analytic uncoded	-3.19
BEC: $\epsilon = 0.20$, BP vs uncoded	12.21

The three channels are not directly comparable on one axis; this table is only a compact way to report order-of-magnitude improvement at representative operating points.

6.6 Code Length and the Capacity Gap

Even at $n = 204$, the BER curves remain noticeably separated from the capacity limits shown in the figures. This is expected, since the implemented codes are regular LDPC constructions with relatively small block lengths, which are not optimized for capacity-approaching performance. In practice, approaching channel capacity requires much larger block lengths ($n \sim 10^3$ – 10^5), irregular LDPC designs with optimized degree distributions, a higher number of decoding

iterations, and advanced decoding strategies such as layered decoding or offset min-sum algorithms. The regular Gallager ensemble used here ($n = 20$ and $n = 204$) is primarily intended for implementation clarity and conceptual understanding, rather than minimizing the gap to capacity.

For block lengths around $n \sim 500$, the same construction principles can be applied, for example with $n = 504$ (or 498 if rounding down), using `ldpc_construction.build_ldpc_for_target_length(500)`. However, without irregular design or further optimization, the improvement toward capacity remains limited.

Practical note: Monte Carlo simulation with iterative decoding is computationally intensive, since each frame requires multiple belief propagation (BP) iterations. The default settings in `experiment.py` are chosen to ensure reasonable runtime on a typical laptop; increasing the number of frames or block length leads to approximately linear growth in runtime. Memory is rarely the bottleneck for these matrix sizes; wall-clock time is the dominant factor.

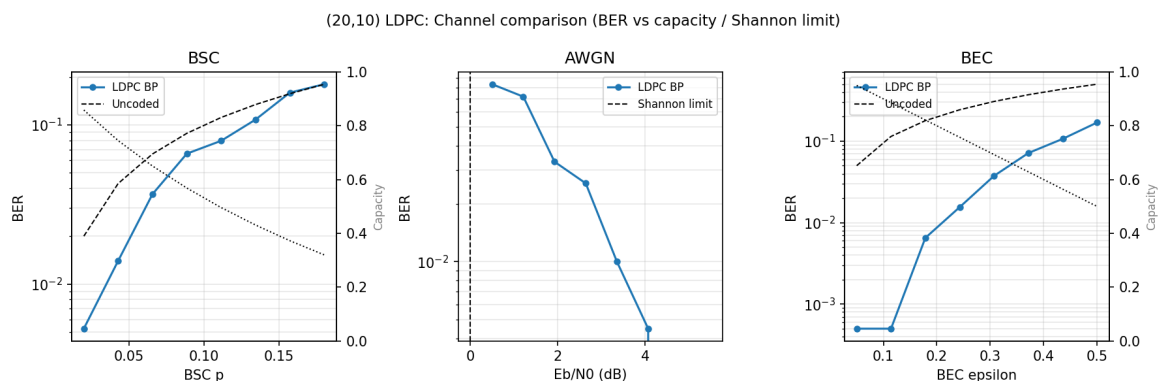


Figure 7 (optional): Channel comparison from `experiments.py` (small (20,10) code).

If present, this extra figure corresponds to the (20,10) LDPC demonstration code generated by `experiments.py`. It is intended for illustration purposes only and is not part of the large-block ($n \geq 200$) performance study.

7. Interactive Demo

The implementation includes an interactive mode where the user can:

1. Enter a custom message (10 binary bits) or generate a random one.
2. See the encoded codeword and verify it satisfies all parity checks.
3. Choose specific bit positions to corrupt.
4. Watch the decoder work step-by-step, showing which bit it flips at each iteration and how many unsatisfied checks remain.
5. See whether the decoder succeeded or failed.

This hands-on approach makes it easy to experiment with different error patterns and develop intuition for how LDPC codes detect and correct errors.

7.1 Sample Interactive Session

```
Enter 10 message bits (0s and 1s), or 'r' for random, 'q' to quit:
> 1010101010

    your message: 1010101010
      codeword: 10111111100101001010
      syndrome: 00000000000000 (valid: True)

Codeword has 20 bits (positions 0-19).
Which bit positions to flip? (e.g. '2 7 15', or 'none'):
> 5 12

    original: 10111111100101001010
  error pattern: 0000010000000100000000
    corrupted: 10111011100111001010

Syndrome is non-zero -- errors detected! Running decoder...

Iter  1: 5 unsatisfied checks -> flip bit 5 (in 3 failing checks)
Iter  2: 3 unsatisfied checks -> flip bit 12 (in 3 failing checks)
Iter  3: syndrome = all zeros -> DONE

SUCCESS -- all 2 error(s) corrected in 3 iteration(s)!
```

To run the interactive demo:

```
python simple_demo.py
```

8. Conclusion

This report demonstrated a complete and correct implementation of LDPC codes covering:

- Gallager's construction for building sparse parity-check matrices.
- Systematic encoding via GF(2) row reduction.
- The Binary Symmetric Channel as a noise model.

- Gallager's bit-flipping algorithm for iterative decoding.

The walk-through with a (20,10) code shows how parity checks detect errors and how bit-flipping can correct a few flips. Section 6 adds Monte Carlo results for a much longer Gallager-regular code ($n = 204$, targeting the $n = 200$ class), with BP and Min-Sum on BSC, AWGN, and BEC. Those curves show real coding gain but still a visible gap to capacity, which is expected until n and the ensemble are scaled up.

The same codebase supports `ldpc_construction.build_ldpc_for_target_length` for n near 500 (e.g. $n = 504$) and the interactive demo remains on $n = 20$ for readability.

References

- [1] R. G. Gallager, "Low-Density Parity-Check Codes," MIT Press, 1963.
- [2] D. J. C. MacKay, "Information Theory, Inference, and Learning Algorithms," Cambridge University Press, 2003. Chapters 25 and 47.
- [3] T. Richardson and R. Urbanke, "Efficient Encoding of Low-Density Parity-Check Codes," IEEE Trans. Information Theory, vol. 47, no. 2, Feb. 2001.